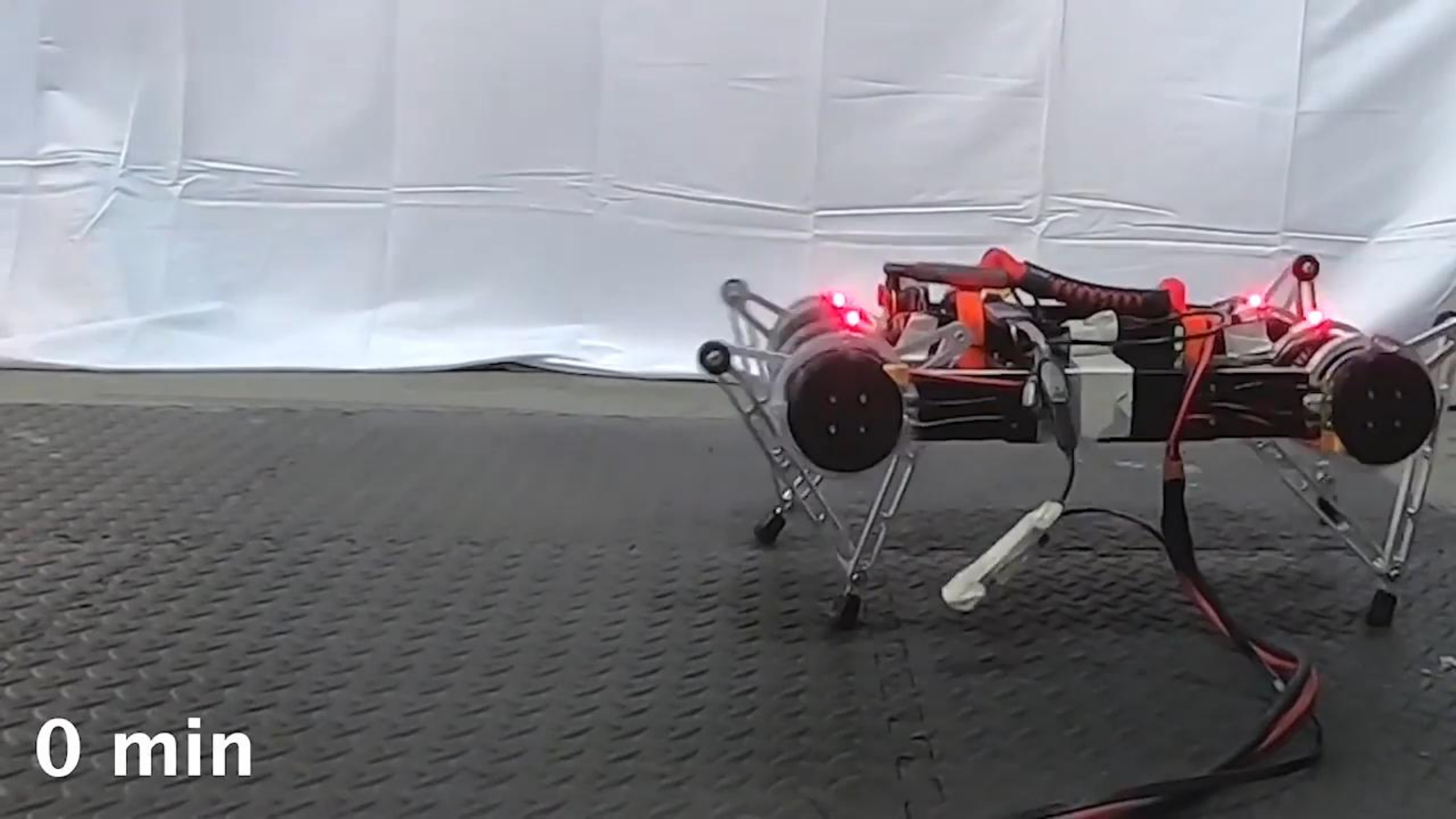


Imitation Learning (2)

[actually about RL]

Dylan Losey

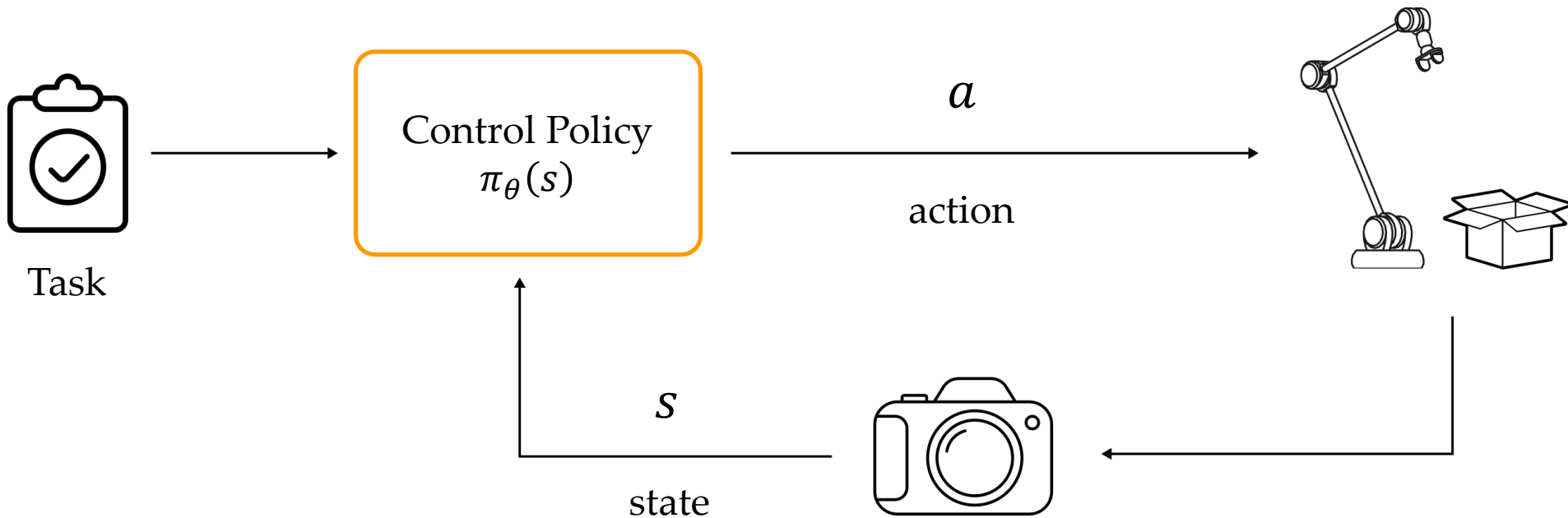


0 min

This Lecture



- (Aside) Explaining reinforcement learning
- Recent directions in imitation learning



- [Task] robot is given a high-level objective, e.g., *pick up the box*
- [State] robot observes the environment (cameras, encoders), and combines the observed information into a state vector s
- [Action] robot chooses actions a to take. These actions could be joint torques τ , or the robot could output desired positions x_d



What is reinforcement learning?



Key Equations

A Markov decision process (**MDP**) is a tuple:

$$\mathcal{M} = \langle S, A, f, r, \gamma \rangle$$

- $s \in S$ are states
- $a \in A$ are actions
- $f(s, a)$ or $f(s^{t+1} \mid s^t, a^t)$ are dynamics
- $r(s)$ is the reward function
- γ is the discount factor

Key Equations

Q-function. $Q(s, a) \rightarrow \mathbb{R}$.

Cumulative discounted reward of taking action a at state s

$$Q(s^t, a^t) = r(s^t) + \gamma \cdot \max_a Q(f(s^t, a^t), a)$$

Reward at s

Long-term reward if we take action a

Can also write as $V(f(s^t, a^t))$

Review

Policy. $\pi(s) = a$.

Mapping from states to actions.

Optimal Policy. Takes actions to maximize the expected discounted sum of rewards, i.e., take actions to **maximize** $Q(s, a)$

Key Equations

Optimal policy. $\pi(s) \rightarrow a$.

Mapping from states to actions that maximizes total reward

$$\pi(s) = \max_a Q(s, a)$$

Take action that maximizes the Q-function

Reinforcement Learning

Given an MDP $\mathcal{M} = \langle S, A, f, r, \gamma \rangle$

The **objective** of reinforcement learning is to learn an **optimal policy** that maximizes the expected discounted sum of rewards.



Soft Actor Critic (SAC)

Initialize Q-function as **neural network**

*Initialize policy $\pi(s)$ as **neural network***

Soft Actor Critic (SAC)

Initialize Q-function as **neural network**

*Initialize policy $\pi(s)$ as **neural network***

While running

- *Evaluate $\pi(s)$ to get action*
- Record the state, action, next state, reward

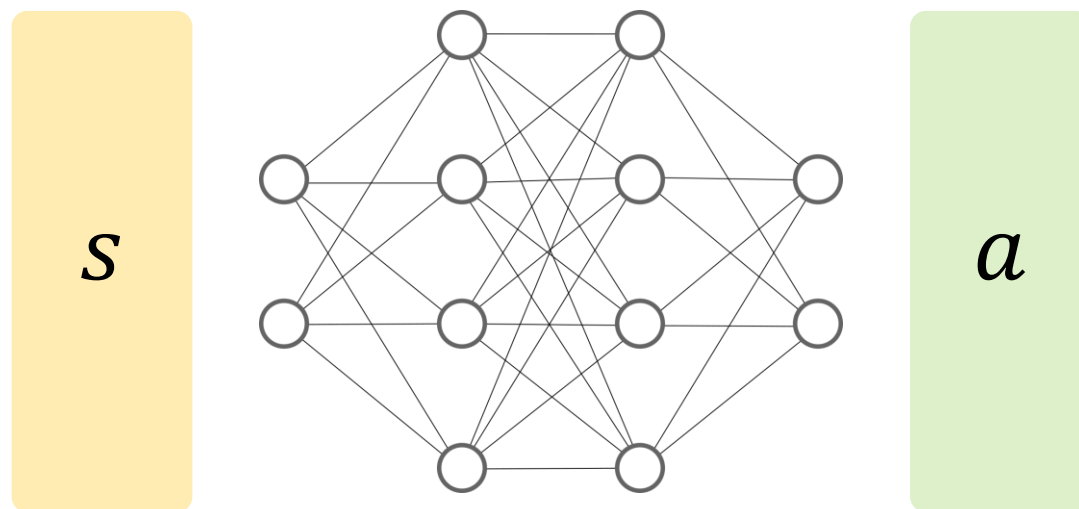
- Update Q-function using data and π

- *Update π to maximize Q-function*



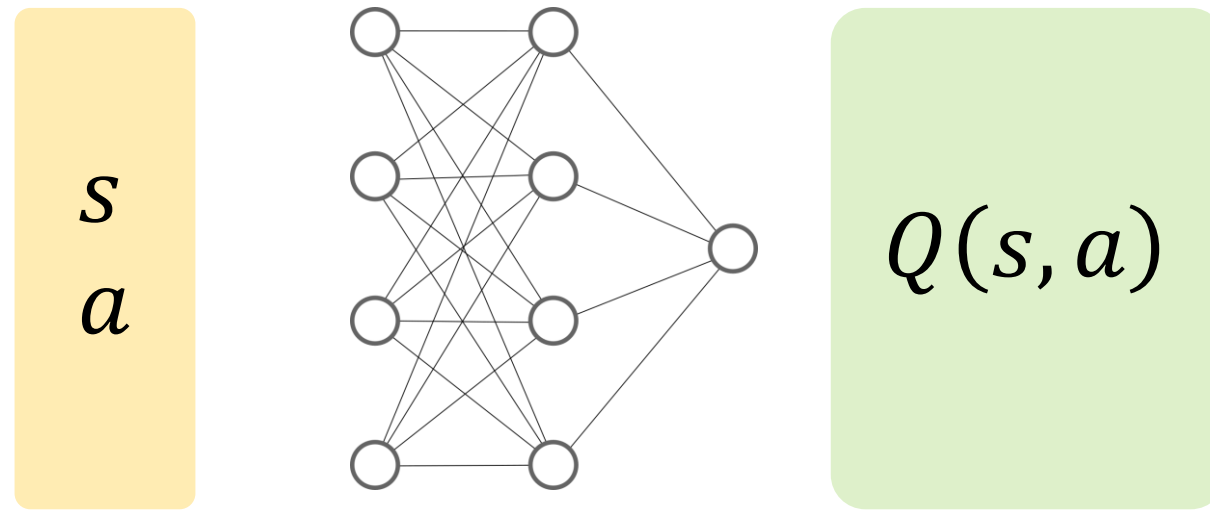
Actor

The actor is the policy $\pi(s) \rightarrow a$



Critic

The critic is the Q-function $Q(s, a) \rightarrow \mathbb{R}$



Critic

The critic is the Q-function $Q(s, a) \rightarrow \mathbb{R}$

We learn **recursively** using two Q-functions, Q and Q'

Given samples s^t, a^t, r^t, s^{t+1} from memory

- $a^{t+1} \leftarrow \pi(s^{t+1})$
- $Q_{target} \leftarrow r^t + \gamma \cdot Q'(s^{t+1}, a^{t+1})$

Use policy to choose next actions.
Get the “correct” Q-value using
the actual rewards, next states,
and next actions.

Critic

The critic is the Q-function $Q(s, a) \rightarrow \mathbb{R}$

We learn **recursively** using two Q-functions, Q and Q'

Given samples s^t, a^t, r^t, s^{t+1} from memory

- $Q_{expected} \leftarrow Q(s^t, a^t)$
- $\mathcal{L}(critic) \leftarrow \|Q_{target} - Q_{expected}\|$

Update Q to match the more accurate estimate $Q_{expected}$

Actor

The actor is the policy $\pi(s) \rightarrow a$

We train the policy to **maximize** the learned Q-function

Given samples s^t, a^t, r^t, s^{t+1} from memory

- $a \leftarrow \pi(s^t)$
- $\mathcal{L}(\text{actor}) \leftarrow -Q(s^t, a)$



Actor

$\pi(s) \rightarrow a$

Proposes actions

Critic

$Q(s, a) \rightarrow \mathbb{R}$

Evaluates actions

Soft Actor-Critic

Initialize policy π and Q-function Q and Q'

- Sample action $a \leftarrow \pi(s)$
- Take action and add s^t, a^t, r^t, s^{t+1} to memory
- **Train the models**

Soft Actor-Critic

Initialize policy π and Q-function Q and Q'

- Sample action $a \leftarrow \pi(s)$
- Take action and add s^t, a^t, r^t, s^{t+1} to memory
- **Train the models**

Sample batch s^t, a^t, r^t, s^{t+1} from memory

- $a^{t+1} \leftarrow \pi(s^{t+1})$
- $Q_{target} \leftarrow r^t + \gamma \cdot Q'(s^{t+1}, a^{t+1})$
- $Q_{expected} \leftarrow Q(s^t, a^t)$
- $\mathcal{L}(critic) \leftarrow \|Q_{target} - Q_{expected}\|$
- $Q' \leftarrow$ weights closer to Q

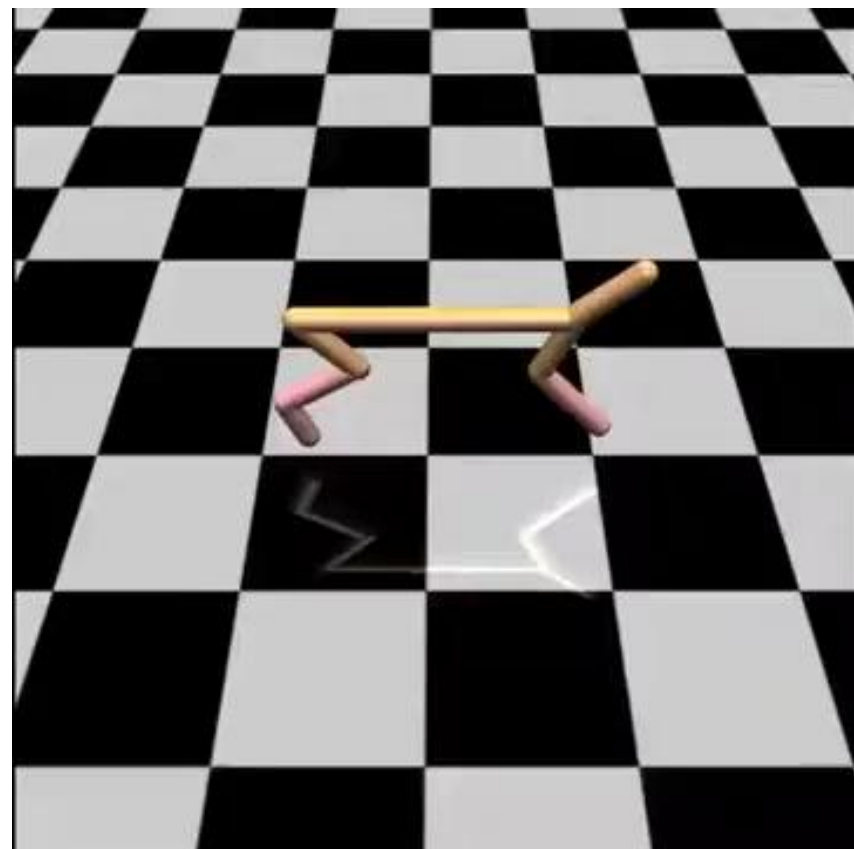
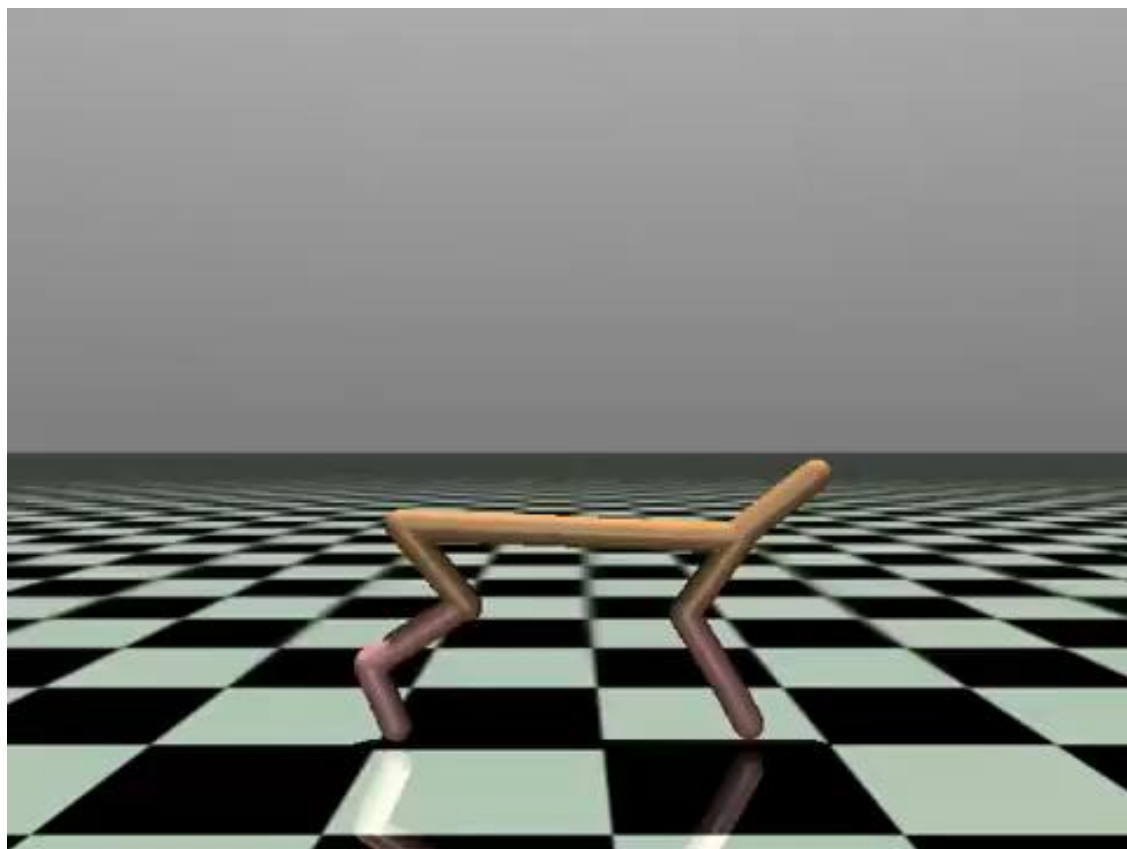
Soft Actor-Critic

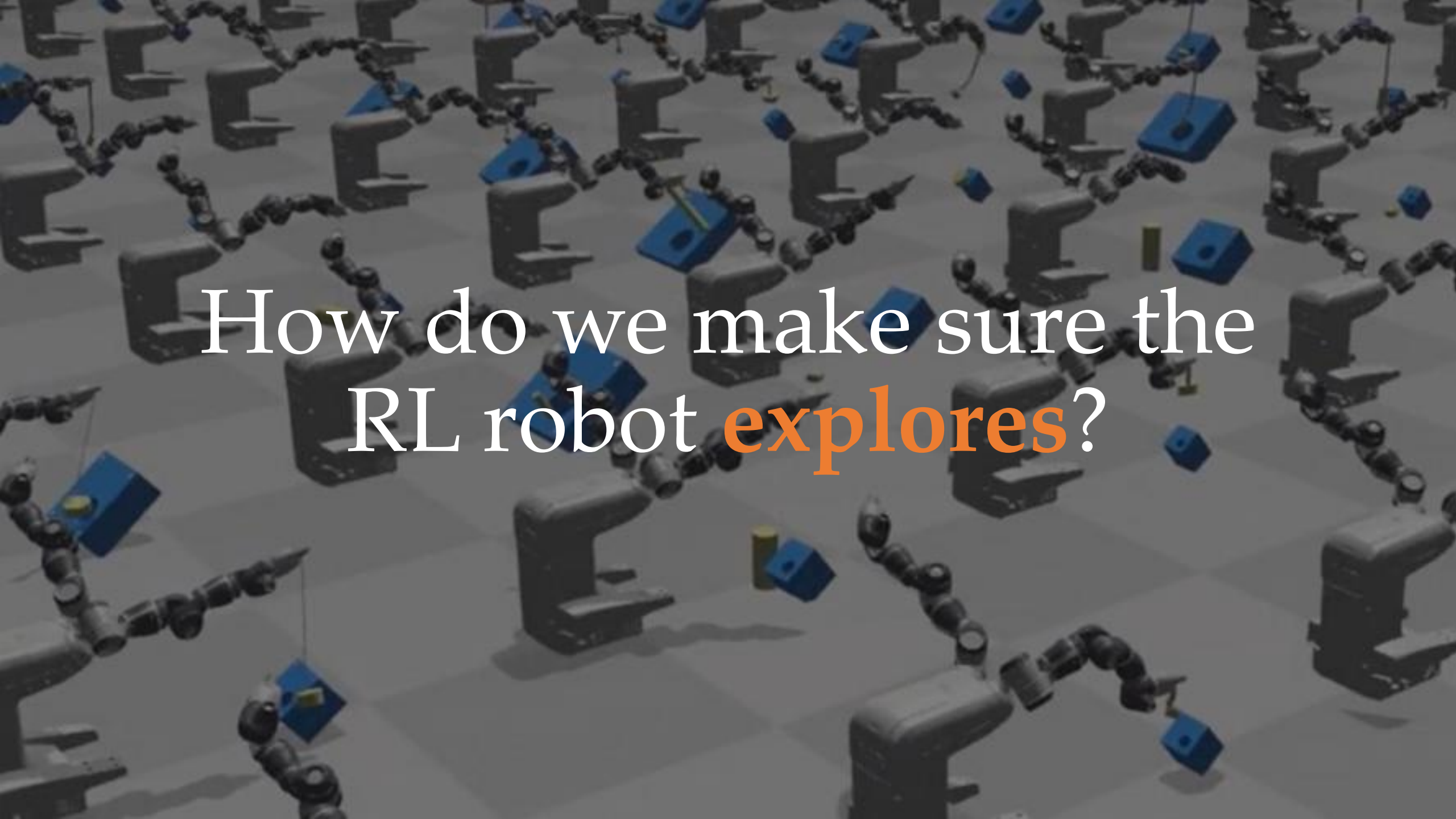
Initialize policy π and Q-function Q and Q'

- Sample action $a \leftarrow \pi(s)$
- Take action and add s^t, a^t, r^t, s^{t+1} to memory
- **Train the models**

Same sample batch s^t, a^t, r^t, s^{t+1} from memory

- $a \leftarrow \pi(s^t)$
- $\mathcal{L}(\text{actor}) \leftarrow -Q(s^t, a)$



The background of the slide is a 3D rendered scene of a robotic environment. It features a light gray and white checkered floor. Numerous robotic arms, each with a black base and a jointed black arm, are positioned across the scene. Some of these arms are holding or interacting with small, blue, rectangular blocks. The scene is populated with many such blocks, some of which are scattered on the floor. The overall lighting is soft, and the scene has a slightly desaturated, blue-gray tint, giving it a technical or scientific feel.

How do we make sure the
RL robot **explores**?

Explore vs. Exploit

Exploit: take the action that maximizes $Q(s^t, a)$

Explore: take a “random” action

- Could encourage robot to take different actions
- Could encourage robot to visit new states
- Objective is to avoid local minima

Explore vs. Exploit

In the SAC framework, we balance explore and exploit by **adding noise** to the robot's policy.

First Pass.

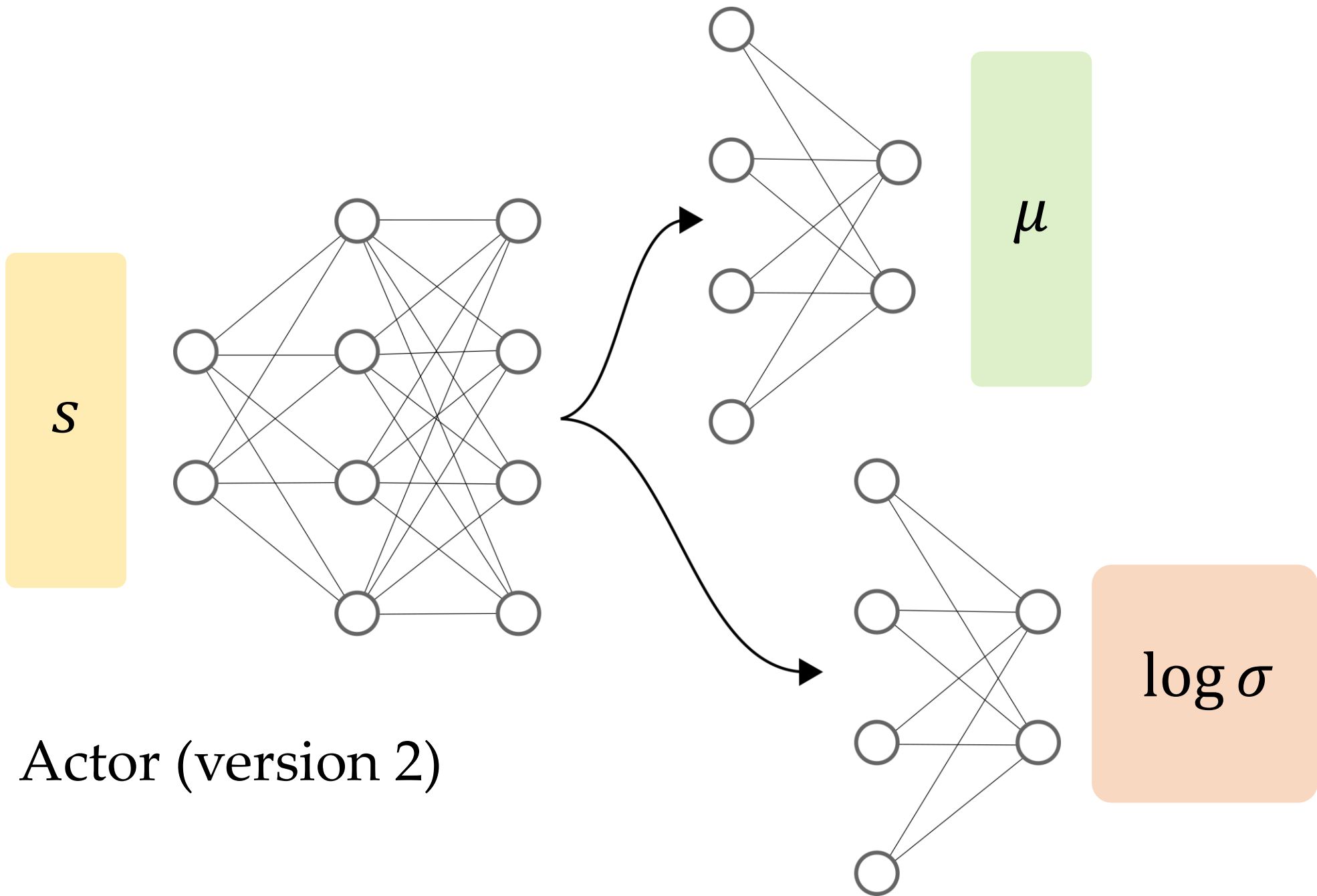
$$a = \pi(s) + \varepsilon, \text{ where } \varepsilon \sim N(0, \sigma^2)$$

Explore vs. Exploit

In the SAC framework, we balance explore and exploit by **adding noise** to the robot's policy.

Second Pass.

$a \sim \pi(s)$, where $\pi(s)$ is a normal distribution



Explore vs. Exploit

The actor is the policy $\pi(s) \rightarrow \mu, \log \sigma$

We train the policy to **maximize** the learned Q-function
and maximize the variance

- $a \sim \pi(s^t) = N(\mu(s), \sigma^2(s))$
- $\mathcal{L}(\text{actor}) \leftarrow -Q(s^t, a) + \alpha \cdot \log \sigma$

α is parameter that we select.

Some versions of SAC learn this parameter

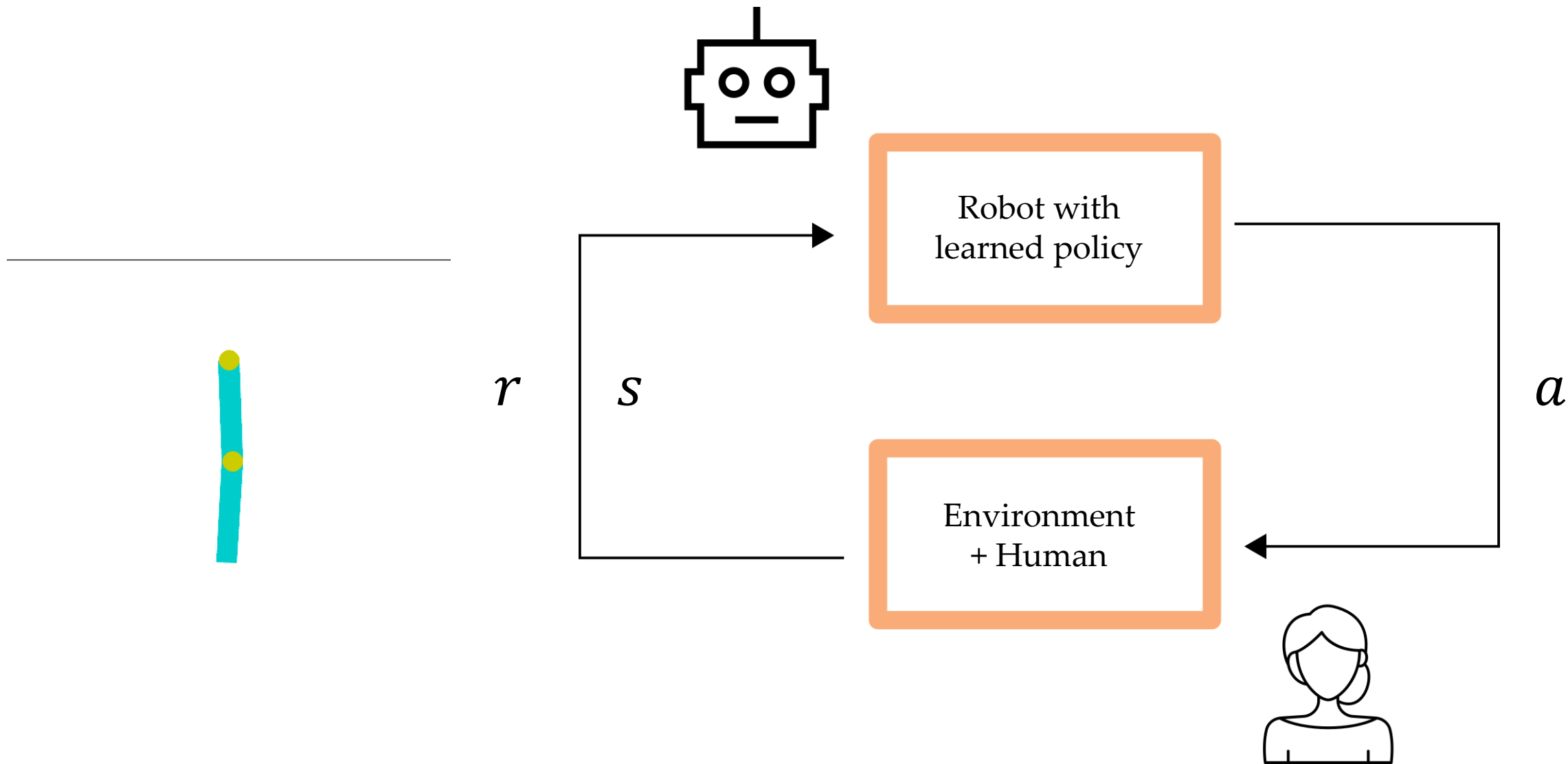
Example

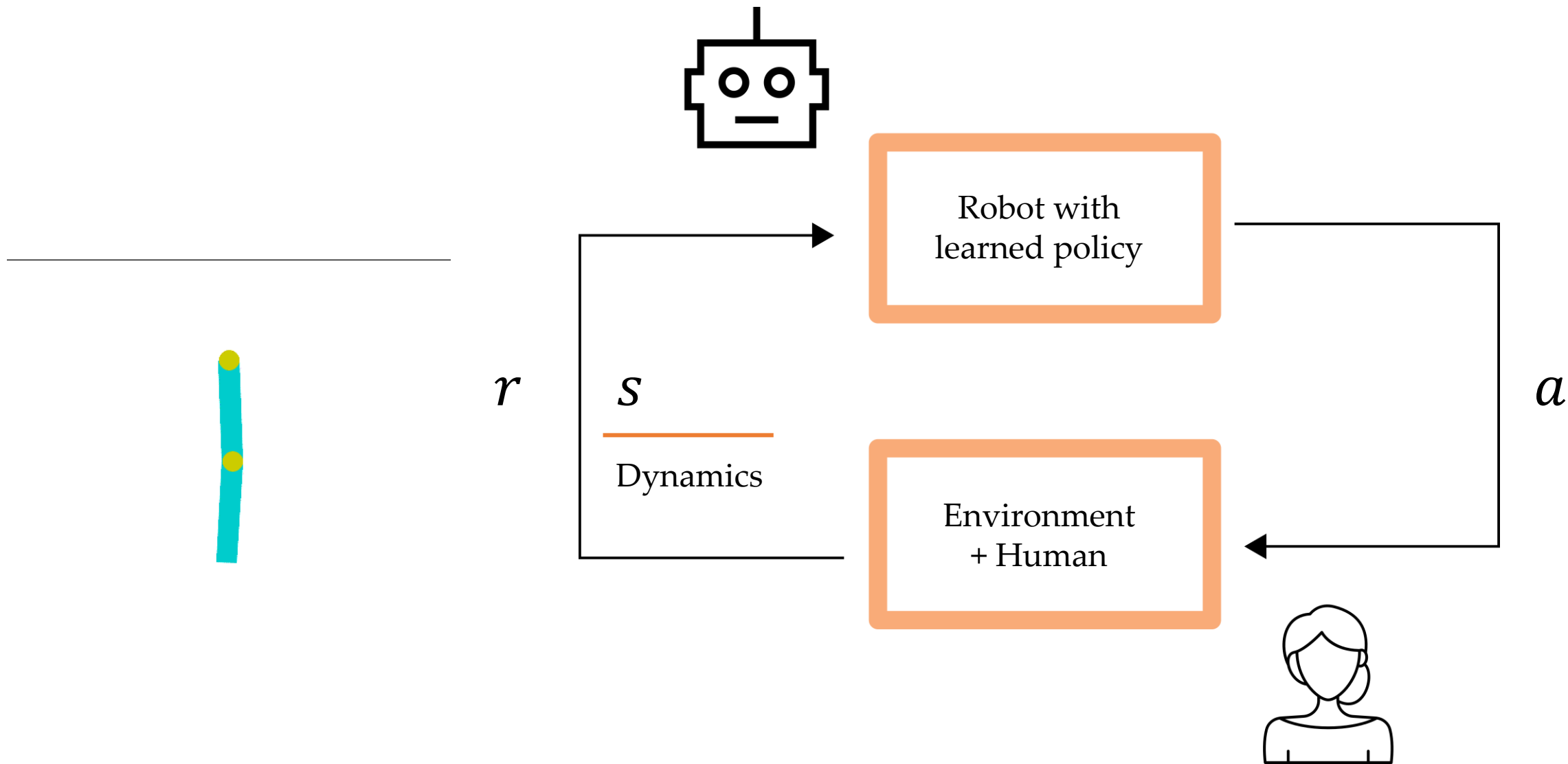


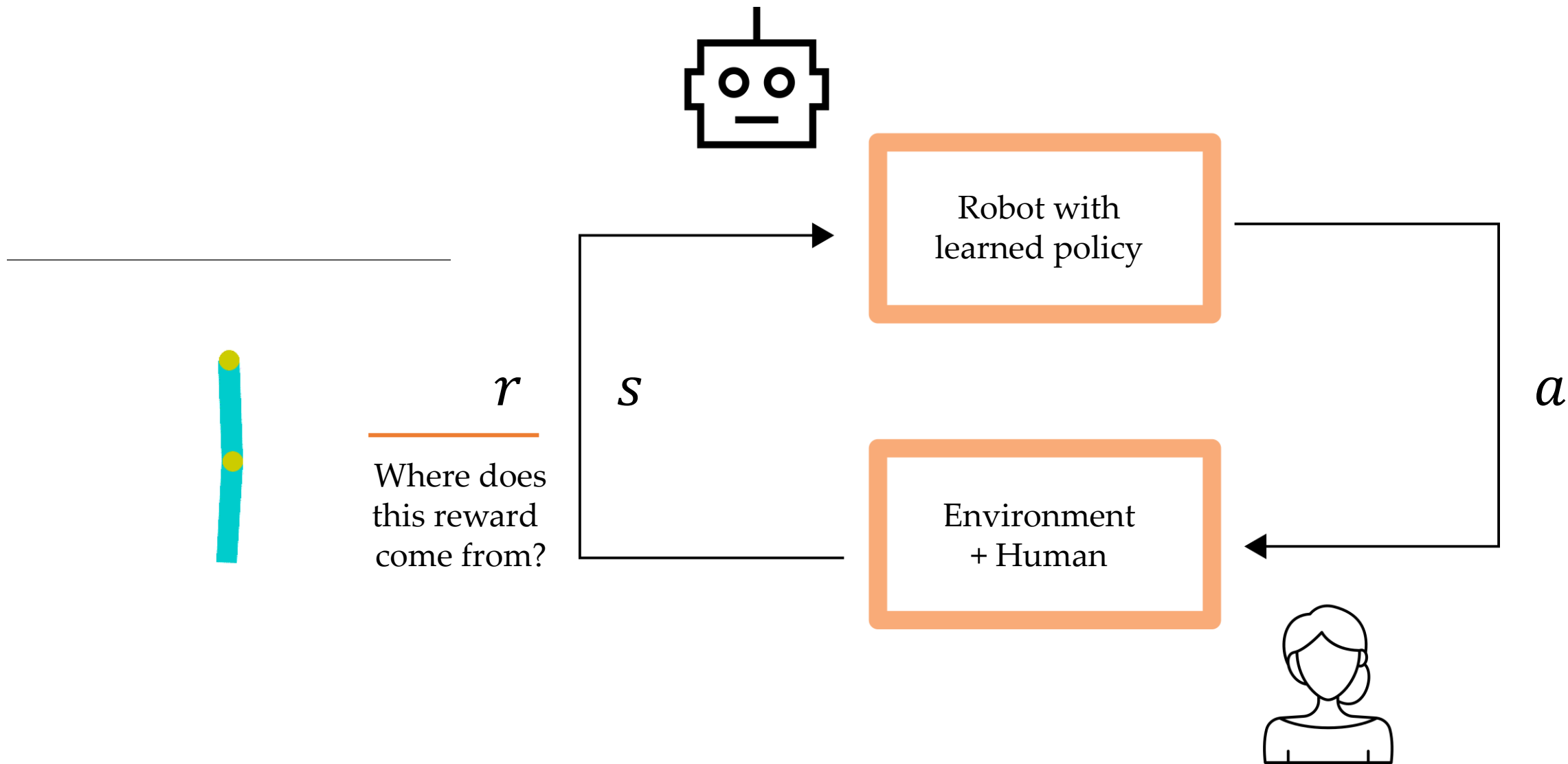


Human-in-the- loop RL









Human-in-the-Loop RL

Emerging field.

One trend is using *feedback from the human* to **learn** the reward function.

Human-in-the-Loop RL

Emerging field.

One trend is using *feedback from the human* to **learn** the reward function.

Robot does not know reward function $r(s)$

While running RL:

- Ask human to assign $r(s)$ at every timestep
- Train RL agent as normal

What's wrong with this first pass solution?

Human-in-the-Loop RL

Emerging field.

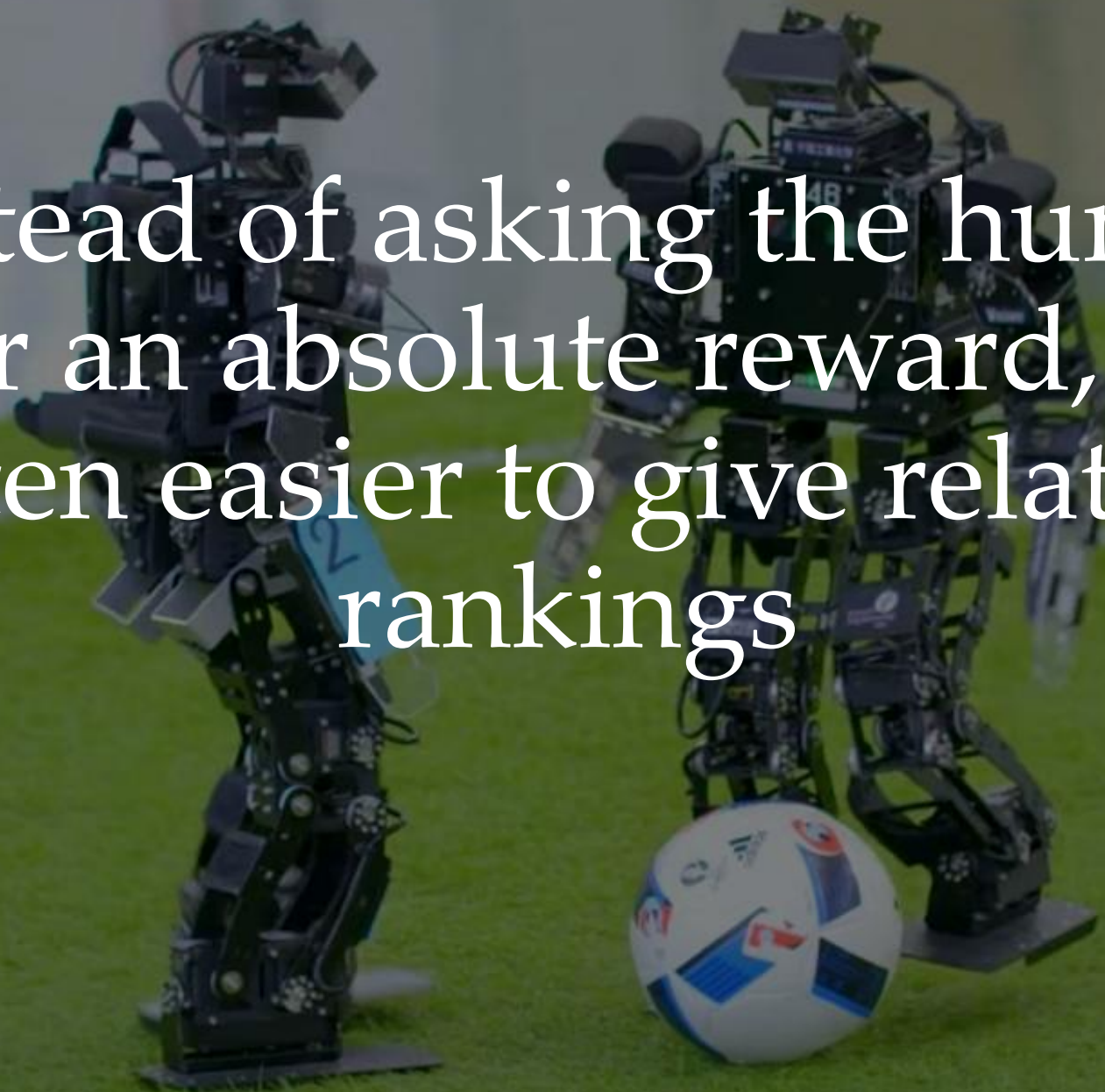
One trend is using *feedback from the human* to **learn** the reward function.

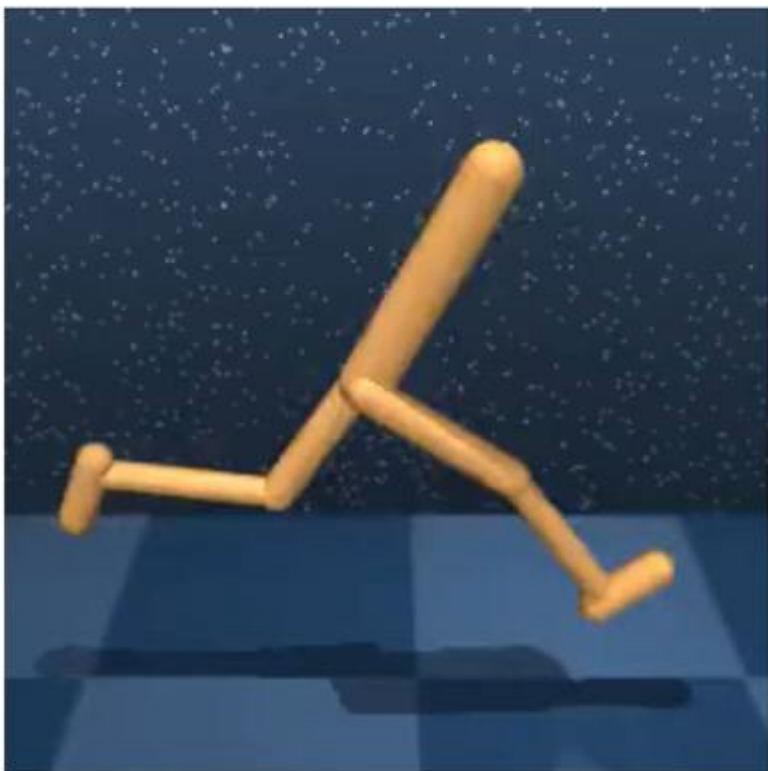
Initialize $r(s)$ as neural network

While running RL:

- Ask human *occasionally* to score the robot's behavior
- **Update $r(s)$ model using human feedback**
- Train RL agent with the learned rewards

Instead of asking the human
for an absolute reward, its
often easier to give relative
rankings

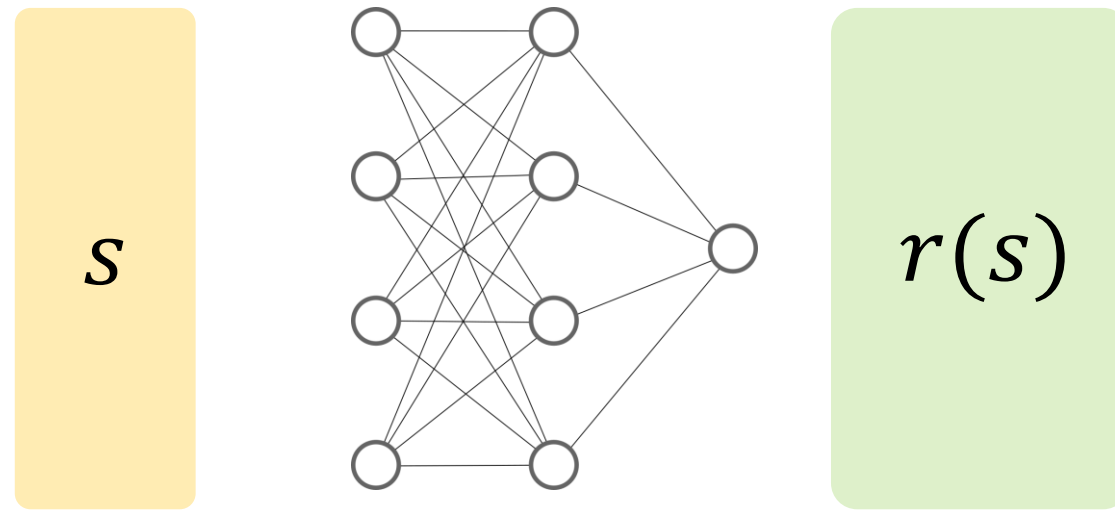




Agent:
Random policy

Reward Model

The reward model is a mapping from states to scalars $r(s) \rightarrow \mathbb{R}$



Reward Model

Let ξ_1 and ξ_2 be two trajectories, $\xi = (s^1, s^2, \dots, s^T)$

Imagine that the human **rank**s $\xi_1 > \xi_2$

How do we learn a reward model from this ranking?

Reward Model

Let ξ_1 and ξ_2 be two trajectories, $\xi = (s^1, s^2, \dots, s^T)$

Imagine that the human **rank**s $\xi_1 > \xi_2$

$$R(\xi_1) = \sum_{s \in \xi_1} r(s^t) > \sum_{s \in \xi_2} r(s^t) = R(\xi_2)$$

The total reward for ξ_1 must be greater than the total reward for ξ_2

Reward Model

The reward model is a mapping from states to scalars $r(s) \rightarrow \mathbb{R}$

We want to **train** the reward model so that:

$$\sum_{s \in \xi_1} r(s^t) > \sum_{s \in \xi_2} r(s^t)$$

Reward Model

The reward model is a mapping from states to scalars $r(s) \rightarrow \mathbb{R}$

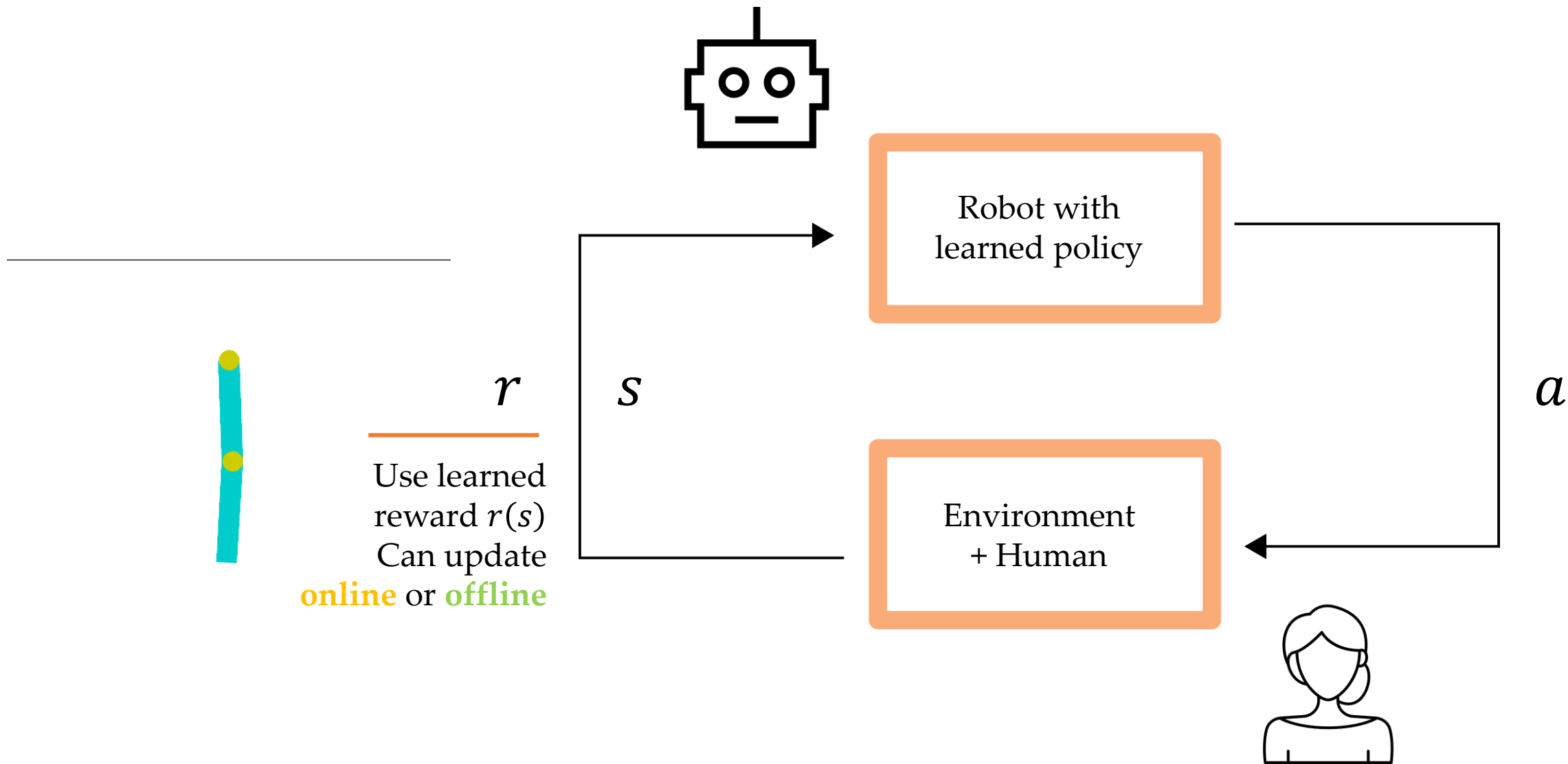
Given a batch of rankings $\mathcal{D}$

For each $(\xi_1, \xi_2) \in \mathcal{D}$:

$$\mathcal{L} = \mathcal{L} + \sum_{s \in \xi_2} r(s^t) - \sum_{s \in \xi_1} r(s^t)$$

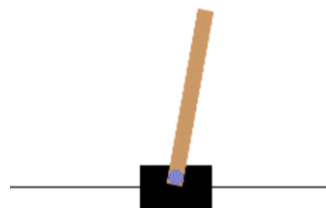
Update reward weights to minimize $\mathcal{L}$

Here we label the trajectory the human prefers as ξ_1

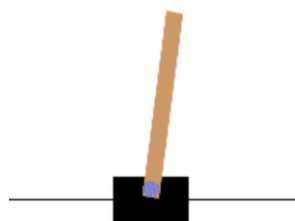


The background image shows two humanoid robots standing on a green artificial turf field. The robot on the left is dark-colored with a blue tag labeled '22' on its leg. The robot on the right is also dark-colored but has a white tag labeled '148' on its chest. A white soccer ball with blue and red patterns is on the ground near the robot on the right. The scene is outdoors with a white fence and trees in the background.

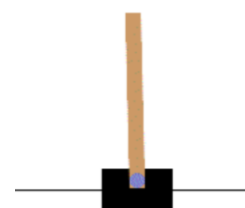
Using this approach, robots
can extrapolate to perform
better than humans!



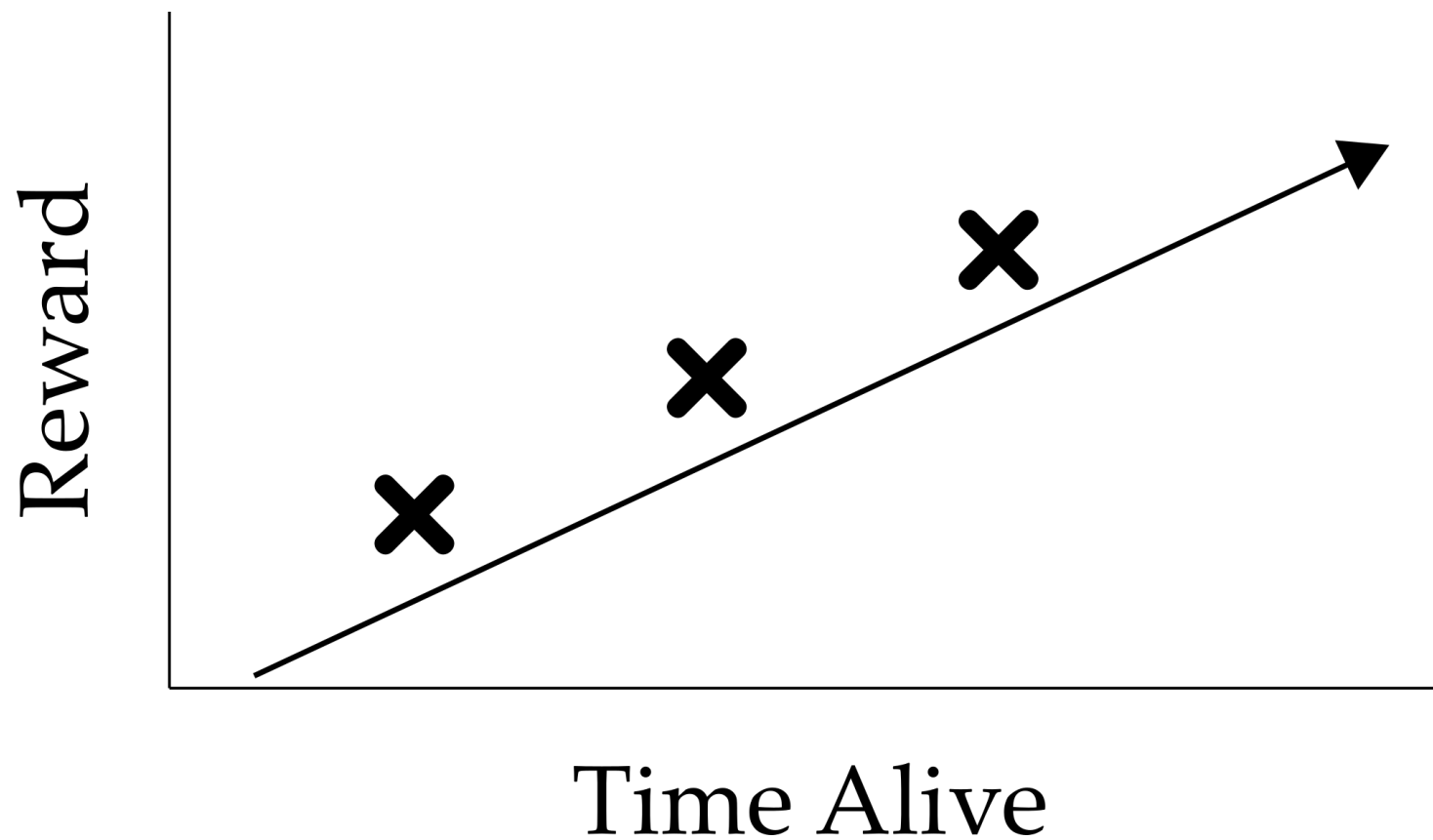
Lasts 10 timesteps



Lasts 30 timesteps

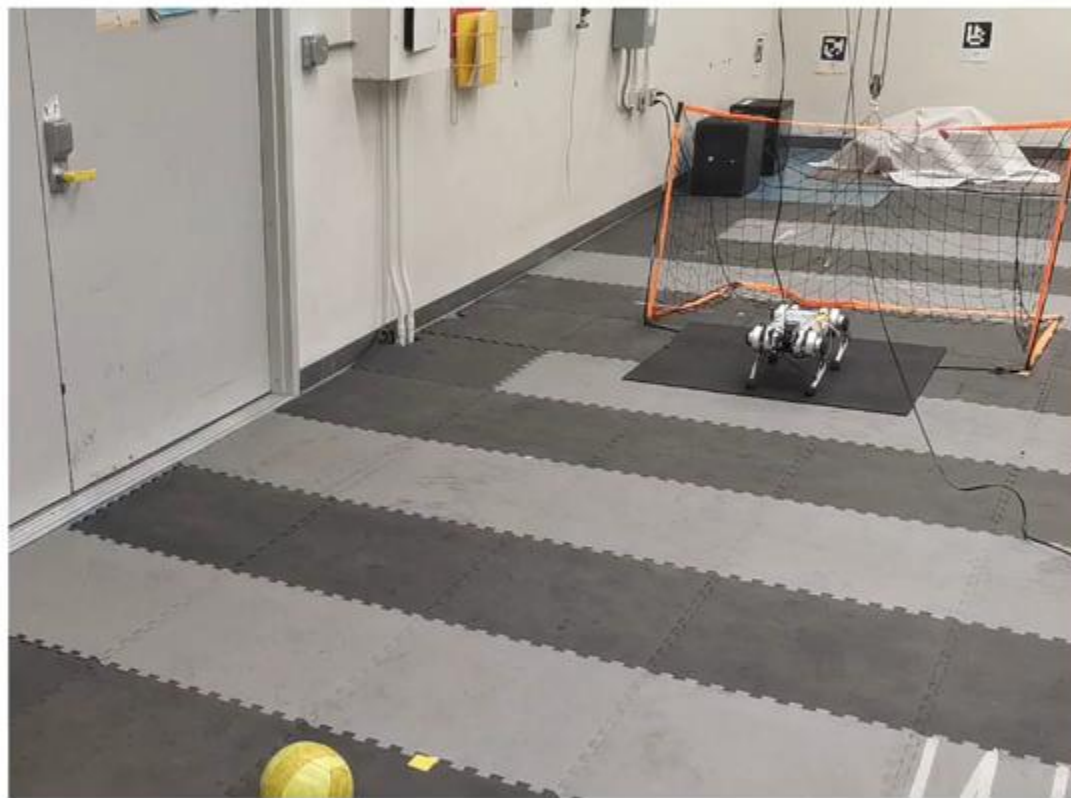


Lasts 80 timesteps



Creating a Dynamic Quadrupedal Robotic Goalkeeper with Reinforcement Learning

Xiaoyu Huang*, Zhongyu Li*, Yanzhen Xiang, Yiming Ni, Yufeng Chi, Yunhao Li, Lizhi Yang, Xue Bin Peng, and Koushil Sreenath



*equal contribution



Berkeley
UNIVERSITY OF CALIFORNIA

hybrid-robotics.berkeley.edu



SIMON FRASER
UNIVERSITY

Offline RL

Given fixed dataset of states, actions, and rewards.

Perform RL, but only on the states in the offline dataset.

- Often adds a constraint so that the learned actions are similar to the demonstrated actions [prevent hallucination]
- Can learn policies that are better than the demonstrator because of the reward signals

Trends in Imitation Learning

- Imitation learning keeps the same behavior cloning structure at a high-level, but now focuses on mapping vision and language to actions.
- Imagine the state is $s = [image, language, robot]$
- Can use foundation models (or variants of these models) to map language and images to meaningful tokens, which then become the state for the downstream policy.
- Can pretrain the foundation models and policy on existing datasets, freeze the foundation model weights, and then fine-tune on smaller datasets (post-training)

This Lecture



- (Aside) Explaining reinforcement learning
- Recent directions in imitation learning

Next Lecture



- Recent trends in imitation learning